

Training SO-100 arm for Pick-and-Place operation using SAC

Aditya Upadhyay(au2414) and Sparsh Vaidya(sv3113)

Abstract

Autonomous pick-and-place is a fundamental manipulation task in robotics. Learning end-to-end policies with reinforcement learning is challenging due to sparse rewards and high-dimensional continuous action spaces. This project investigates the Soft Actor-Critic (SAC) algorithm combined with curriculum learning for training a single robotic manipulation policy. We adopt a progressive task decomposition strategy: train one SAC agent sequentially on four subtasks (object approach, gripper closure, lifting, and placement), fine-tuning the policy on each phase with task-specific reward shaping. This approach allows the model to reuse learned representations and control primitives across subtasks. Experiments are conducted in simulation on an SO-100 robotic arm in MuJoCo. The same SAC model achieves 100% success on the complete pick-and-place task (binning a cube) after approximately 10M total environmental steps. We demonstrate that curriculum based progressive training significantly accelerates the convergence compared to the end to end approaches. Our results validate SAC's effectiveness for continuous control in robotic manipulation and show that task decomposition with knowledge transfer enables efficient learning of complex multi-phase manipulation behaviors.

1. INTRODUCTION

Autonomous robotic manipulation is a foundational problem in robotics, with applications spanning industrial automation, warehouse logistics, and collaborative robotics. Pick-and-place tasks, where a robot must locate, grasp, and reposition objects, represent a standard benchmark for evaluating manipulation capabilities. While traditional approaches rely on hand-crafted control policies and trajectory planning, reinforcement learning (RL) offers an alternative: learn manipulation strategies directly through interaction and reward-based feedback.

Continuous control tasks like robotic manipulation present specific challenges for RL algorithms. The action space is continuous and high-dimensional, exploration is expensive (failed grasps are costly), and reward signals are often sparse or delayed. The Soft Actor-Critic (SAC) algorithm has shown promise in these settings due to its stability and entropy-regularized exploration, making it well-suited for robotic control.

However, learning an end-to-end pick-and-place policy from scratch is notoriously difficult. The task involves multiple distinct phases: object approach, gripper closure, lifting, and placement, each with different control requirements and failure modes. We adopt a curriculum learning approach: train SAC on each subtask sequentially, with task-specific reward functions, rather than attempting to learn the full pipeline end-to-end. This reduces exploration burden and provides clearer learning signals at each stage.

This project evaluates SAC within this curriculum learning framework on a simulated SO-100 robotic arm in MuJoCo. We trained four separate policies for each task phase and measure success rates, sample efficiency, and convergence behavior. Our results demonstrate that SAC, combined with curriculum-based task decomposition, achieves 100% success on the complete pick-and-place task after approximately 10M environment steps of training. The progressive training strategy enables the agent to learn stable and reusable control policies across subtasks, with each phase building on learned representations from the previous stage.

These results validate the effectiveness of curriculum learning in reducing the complexity of end-to-end training and show that SAC is well-suited for continuous control in robotic manipulation. We further demonstrate that task-specific reward shaping and progressive fine-tuning significantly improve convergence and final performance compared to naive end-to-end approaches. This work provides insights into efficient policy learning for multi-phase manipulation and motivates further investigation into curriculum design and knowledge transfer in robotic control.

The remainder of this report is organized as follows. Section 2 formulates the problem statement and defines the task setting. Section 3 describes our proposed solution and learning approaches, including

the environment setup and reward design. Section 4 presents the experimental results and analysis. Section 6.4 discusses limitations and future directions.

1.1. Motivation

Despite recent advances in reinforcement learning for robotic control, learning robust manipulation policies remains a significant open challenge, particularly in real-world or high-fidelity simulated environments. End-to-end training of manipulation pipelines is notoriously sample-inefficient: sparse reward landscapes and the compounding difficulty of multi-phase tasks mean that naive RL agents rarely converge to reliable policies within practical compute budgets. While SAC has demonstrated strong performance on continuous control benchmarks, its application to structured, multi-stage tasks such as pick-and-place has received comparatively little systematic study. Furthermore, the role of curriculum design, specifically how task decomposition and phase-specific reward shaping interact with entropy-regularized policy optimization, remains poorly understood. This work is motivated by the need to establish whether curriculum learning can meaningfully close the gap between SAC's theoretical promise and its practical performance on multi-phase manipulation tasks. By evaluating SAC under a principled curriculum framework on a simulated SO-100 arm, we aim to quantify the benefits of progressive training in terms of sample efficiency, convergence stability, and final task success, and to provide actionable insights for deploying similar approaches in physical robotic systems.

1.2. Contribution

The main contributions of this work are:

- A fully configured RL training pipeline for the SO-100 arm using SAC and Stable Baselines 3.
- A reward function analysis identifying the cause of incomplete contact learning.
- An interactive visualisation and reward-probe toolkit for diagnosing agent behaviour.
- Recommendations for reward redesign and extended training.

2. PROBLEM STATEMENT

This project addresses the problem of training an autonomous manipulation policy for a simulated SO-100 robotic arm to perform a pick-and-place task: detecting and grasping a red cube positioned on a table surface and successfully relocating it into a target bin. The task is implemented entirely within a MuJoCo simulation environment, removing real-world hardware constraints and allowing for rapid,

repeatable experimentation. Rather than relying on complex architectural innovations or imitation learning from demonstrations, this work deliberately constrains the methodology to a single algorithm, Soft Actor-Critic, and investigates how far careful reward function design alone can drive policy performance.

The core challenge lies in the fact that pick-and-place is a multi-phase task: the agent must sequence approach, grasp, lift, transport, and release behaviours in the correct order. The primary question this project seeks to answer is therefore whether SAC, guided solely by a well-shaped reward signal and a curriculum of incrementally harder subtasks, is sufficient to produce a reliable, generalisable pick-and-place policy, without the need for hand-crafted controllers, trajectory planners, or privileged state information beyond what a standard simulation environment exposes.

2.1. Task Definition

Robotic manipulation is one of the most challenging open problems in autonomous systems research. Even tasks that are trivial for humans, reaching for an object, grasping it, and moving it to a target location, require a robot to solve a high-dimensional continuous control problem under physical constraints and sensory uncertainty.

Reinforcement learning offers a promising framework for robot control: rather than hand-coding a controller for every possible configuration, the agent learns a policy by interacting with its environment.

We define the manipulation problem as a Markov Decision Process (MDP):

$$M = (S, A, R, P, \gamma),$$

where:

- $S \subset \mathbb{R}^{12}$ is the state space,
- $A = [-1, 1]^6$ is the continuous action space,
- $R : S \times A \rightarrow \mathbb{R}$ is the reward function defined in Section 3.2,
- P is the transition dynamics governed by MuJoCo physics at 50 Hz,
- $\gamma = 0.99$ is the discount factor.

2.2. State Observation

Recent reinforcement learning algorithms such as Soft Actor-Critic (SAC) have demonstrated impressive results on continuous-action robotic benchmarks.

This project applies these ideas to the SO-100 robot arm, a six-joint manipulator originally designed for teleoperation tasks. Using MuJoCo as the physics backend and the `gym-so100` package as the environment interface, we train an SAC agent to perform three manipulation tasks of increasing complexity.

At each timestep t , the agent receives a 12-dimensional observation vector:

$$s_t = [p_{\text{cube}}, p_{\text{bin}}, p_{\text{ee}}, q] \in \mathbb{R}^{12} \quad (1)$$

where $p_{\text{cube}}, p_{\text{bin}}, p_{\text{ee}} \in \mathbb{R}^3$ are the 3-D Cartesian positions of the cube, bin centre, and end-effector respectively, and $q \in \mathbb{R}^6$ contains the current joint angles.

2.3. Action Space

The action at timestep t , $a_t \in [-1, 1]^6$, is a normalised joint position command. Before being sent to the simulator, each component is un-normalised to the corresponding real joint range.

Table 1. Joint Ranges and Normalisation

Index	Joint	Min (rad)	Max (rad)
0	Waist	-1.92	+1.92
1	Shoulder	-3.32	+0.174
2	Elbow	-0.174	+3.14
3	Forearm Roll	-1.66	+1.66
4	Wrist Rotate	-2.79	+2.79
5	Gripper	-0.174	+1.75

2.4. Episode Structure

Each phase of the curriculum is trained in its own isolated environment, with initial conditions, termination criteria, and success rewards tailored to the specific subtask. At the start of every episode, the arm is reset to a fixed home pose and the red cube is spawned at a uniformly random position within:

$$x \in [-0.25, -0.15] \text{ m}, \quad y \in [0.30, 0.60] \text{ m}, \quad z = 0.05 \text{ m},$$

ensuring the policy is exposed to varied object configurations throughout training.

The four subtask environments differ in their terminal reward magnitudes, reflecting the relative difficulty and importance of each phase, and in their success conditions, which become progressively more demanding across the curriculum. An episode terminates either upon satisfaction of the phase-specific success condition or upon reaching the maximum step budget T , which varies per phase depending on the simulation framerate. During visualised evaluation runs, this corresponds to approximately 1 to 1.5 minutes of simulated time in practice.

This per-phase episode structure prevents the agent from conflating subtask objectives and ensures that reward signals remain dense and informative relative to the complexity of the behaviour being learned at each stage.

3. SOLUTION AND APPROACHES

3.1. Environment Architecture

The simulation environment is built on a modified version of the open-source `gym-so100-c` repository. This repository was adapted to support the training pipeline developed in this work. The physics in this environment are provided by MuJoCo, accessed through the `dm_control` wrapper. The robot and scene geometry are defined in the XML model `so100_transfer_cube.xml`. This setup is well suited for simulating contact dynamics and numerical integration, which are both important for accurate reward computation during grasping and placement.

The simulated platform is the SO-100, a 6-DOF robotic arm consisting of five arm joints and one gripper. The arm operates in a six-dimensional control space. The observation space is a 15-dimensional vector that includes the cube position, bin position, end-effector position, and current joint angles. This state representation was intentionally kept simple and compact. Raw pixel inputs were not used. This choice makes policy learning way more tractable under the SAC framework.

Several issues were encountered during environment setup and subsequently resolved. A conflict between Wayland and GLX required switching the OpenGL backend to OSMesa by setting `MUJOOCO_GL=osmesa`. This enabled stable offscreen rendering. In addition, the `gym-so100` package was incorrectly installed, which was resolved by reinstalling the environment directly from the local source directory. A bug in the policy visualisation pipeline was also identified, where rendering was being sent to a different environment instance. This was fixed by introducing a `get_inner_env` accessor to ensure that the correct environment instance is consistently used during evaluation.

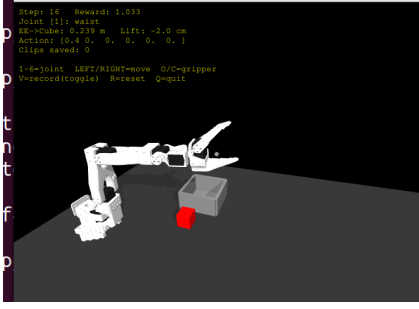


Figure 1. MuJoCo simulation environment of the SO-100 robotic arm performing the pick-and-place task.

3.1.1. Physics Simulator

Simulation is provided by MuJoCo (Multi-Joint dynamics with Contact), accessed through the `dm_control` Python wrapper. The robot, cube, table, and bin are defined in an XML asset file (`so100_transfer_cube.xml`). MuJoCo handles rigid-body dynamics, joint actuation, and contact detection at a timestep of 20 ms (50 Hz). The simulation runs headlessly using the OSMesa software renderer (`MUJOOCO_GL=osmesa`), which avoids conflicts between MuJoCo’s EGL context and the `pygame` display used for visualisation. The custom `gym-so100` environment wraps `dm_control` into a standard Gymnasium interface.

3.1.2. State Space

The agent observes a 15-dimensional continuous state vector referred to as `so100_state`.

All positions are in meters. Joint angles are represented in the normalised action range rather than raw radians. The state does not include velocity or gripper force; the agent must infer contact implicitly from position changes. The bin position is fixed across episodes, so the agent effectively learns it as a constant landmark.

Table 2. Observation Space Components

Component	Dims	Description
Cube position	3	XYZ coordinates of cube center
Bin position	3	XYZ coordinates of bin center
End-effector position	3	XYZ coordinates of <code>ee_site</code>
Joint angles	6	current pos. of all 6 actuated joints

3.1.3. Action Space

Actions are 6-dimensional continuous vectors in the range $[-1, 1]^6$, one value per joint: [waist, shoulder, elbow, forearm_roll, wrist_rotate, gripper]. The environment maps these normalised values to actual joint angle targets via `unnormalize_so100()`, which scales each dimension to the joint’s physical limits. The gripper dimension maps -1 (fully closed) to $+1$ (fully open). Actions are applied as position targets, so the arm moves toward the commanded configuration at each MuJoCo step rather than receiving instantaneous velocity impulses.

3.1.4. Observation Normalization

Raw observations are normalised online using `VecNormalize` from `Stable-Baselines3`. `VecNormalize` maintains a running mean and standard deviation for each observation dimension and standardises inputs to have approximately zero mean and unit variance. This prevents larger-magnitude dimensions (e.g., joint angles in radians) from dominating the neural network inputs. Reward normalisation is disabled (`norm_reward=False`) to keep the reward signal directly interpretable during debugging.

The normalisation statistics are saved alongside each model checkpoint and must be loaded together with the policy at evaluation time to ensure consistent scaling.

3.2. Task Design & Reward Functions

A central contribution of this work is the design of the reward functions that guide policy learning across each phase of the curriculum. Since all tasks share the same robot, physics engine, observation space, and action space, the reward function is the sole mechanism through which the agent’s objective changes between phases, making its design critical to training success.

Rather than relying on a single monolithic reward for the full pick-and-place pipeline, three tasks of increasing complexity were constructed, each isolating a distinct behavioural milestone: reaching and touching the cube, grasping and lifting it, and finally transporting and releasing it into the target bin. The design process was iterative; not all reward formulations succeeded on the first attempt, and the failures proved as informative as the successes. The following subsections document the reward structure, termination conditions, and training outcomes for each task, along with the diagnostic reasoning that motivated revisions where necessary.

3.2.1. Contact Detection

All reward functions rely on MuJoCo’s contact detection, not any external sensor. At each step, the physics engine reports all active contact pairs as `(geom_1, geom_2)` tuples. The gripper jaw pads are named `fixed_jaw_pad_1` through `fixed_jaw_pad_4` and `moving_jaw_pad_1` through `moving_jaw_pad_4` — eight geoms in total.

A cube contact is registered when any of these eight geoms appears alongside the `red_box` geom in a contact pair:

```
touch_gripper = any(
    (g1 in FINGERTIP_GEOMS and g2 == "red_box") or
    (g2 in FINGERTIP_GEOMS and g1 == "red_box")
    for (g1, g2) in all_contact_pairs
)
```

Table contact is detected similarly by checking for the pair `(“red_box”, “table”)`. Bin containment is checked geometrically against the bin’s axis-aligned bounding box (half-width 6 cm in XY, height 3 cm), which is pre-computed from the `bin_center` site at each episode reset.

3.2.2. Task 1: SO100TouchCube

Goal: Move the end-effector to touch the cube. Episode ends when the gripper makes contact within 5 cm of the cube.

Max episode steps: 300

Success condition:

`touch_gripper & ee_cube_dist < 0.05`

Reward function — multi-tier distance shaping:

The reward uses nested distance bands, each providing a progressively stronger gradient as the end-effector approaches the cube.

Table 3. Distance-based reward shaping

Distance threshold	Reward formula	Max value
$dist < 0.7$ m	$0.1 \times (1 - d/0.7)$	0.10
$dist < 0.5$ m	$0.2 \times (1 - d/0.5)$	0.20
$dist < 0.3$ m	$0.5 \times (1 - d/0.3)$	0.50
$dist < 0.1$ m	$1.0 \times (1 - d/0.1)$	1.00
$dist < 0.05$ m	$2.0 \times (1 - d/0.05)$	2.00
Gripper contact	+1.0 bonus	–
Step penalty	–0.2 per step	–
SUCCESS	return 4.0, episode ends	–

Table 4. GraspCube reward structure

Stage	Condition	Reward	Max
1 – Approach	$\text{dist} < 0.7 \rightarrow 0.05 \text{ m}$	same distance bands as TouchCube	1.5
2 – Touch	<code>touch_gripper</code>	+1.0	1.0
3 – Lift (base)	<code>touch_gripper</code> && $\neg \text{touch_table}$	+1.0	1.0
3 – Lift (scaled)	height above table	$0.5 \times \min(\text{Lift}/0.04, 1.0)$	0.5
Step penalty	always	−0.05	–
SUCCESS	all conditions met	return 4.0	–

The bands are computed using a `max()` selection so that only the tightest active band contributes; there is no double-counting. The step penalty of -0.2 discourages the policy from lingering without progress. The maximum non-success reward per step is approximately $2.0 + 1.0 - 0.2 = 2.8$, which is below the success signal of 4.0.

Training results:

- **Run 1** (300k steps, 2 envs): $\text{ep_rew_mean} \approx 0.87$, success rate = 0%. The robot approaches but never discovers contact.
- **Run 2** (resumed to 1M steps, $8 \rightarrow 4$ envs): $\text{ep_rew_mean} \approx 150$, success rate $\approx 38\text{--}40\%$, $\text{ep_len_mean} \approx 213$. The policy reliably approaches and touches the cube. This model was used as a warm start for all subsequent tasks.

3.2.3. Task 2: SO100GraspCube

Goal: Pick up the cube — touch it, close the gripper, and lift it at least 4 cm off the table. A new reward stage for lifting was added on top of the TouchCube approach shaping.

Max episode steps: 400

Success condition:

$$\text{touch_gripper} \wedge \neg \text{touch_table} \wedge \text{lift_height} \geq 0.04$$

Reward function — 4 stages:

The approach shaping maximum was reduced from 2.0 (TouchCube) to 1.5 at the closest band (5 cm) to leave head-room for lift bonuses without exceeding the success signal. The step penalty was reduced to -0.05 (from -0.2 in TouchCube) to avoid discouraging the policy from hovering near the cube while attempting to close the gripper.

The cube always starts at $z = 0.05$ m (constant), so

```
lift_height = cube_z - 0.05.
```

The $\neg \text{touch_table}$ condition is critical: it prevents the policy from claiming lift reward while the cube is still resting on the surface.

Training results:

Resumed from TouchCube 1M checkpoint. Trained for approximately 2.3M additional steps. `ep_rew_mean` rose to 260+ but success rate remained 0% throughout. The policy learned to approach the cube consistently (inheriting TouchCube skill) and to oscillate the gripper near it, but never discovered the coordinated sequence of closing the gripper while the cube is between the fingers and then lifting. The GraspCube task was ultimately judged as a failed intermediate step.

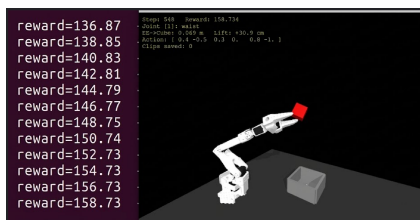


Figure 2. Demonstration of grasping the cube

3.2.4. Task 3 – SO100PickAndPlace v1 (Failed Attempt)

Goal: Full pick-and-place pipeline — approach, grasp, lift, transport to bin, release inside bin.

Max episode steps: 500

Success condition:

cube released (no gripper contact) $\wedge$ cube inside bin AABB

The first version of this reward was designed with scaled-down approach shaping (max 0.6 instead of 1.5) to leave more budget for transport and drop rewards. It was warm-started from the GraspCube 2.3M checkpoint.

Reward structure (v1):

Table 5. Pick-and-place reward structure (v1)

Stage	Max reward
1 — Approach	0.6 (reduced from 1.5)
2 — Touch	+0.4
3 — Lift	+0.4 to +0.7
4 — Transport (bin proximity)	+0 to +0.8
5 — Over bin	+0.4
Step penalty	−0.05

Result: Catastrophic failure. The policy appeared to move randomly; it no longer approached the cube despite inheriting a model that had trained for millions of steps. Diagnostic inspection revealed that the end-effector consistently reached approximately 0.15 m from the cube before retreating.

Root cause — reward scale mismatch: The warm-started policy’s value function had been calibrated on an approach gradient that maxed at 1.5. Reducing the approach reward ceiling to 0.6 meant the policy’s learned value estimates for approach states were now far too high relative to actual rewards received. The policy “believed” approach was worth more than it was, causing unstable Q-value updates that degraded approach behaviour. This is a form of catastrophic forgetting triggered not by changing the task structure but by rescaling the reward.

Training was stopped at approximately 2.67M steps with $\text{ep_rew_mean} \approx 99$ and success rate = 0%.

3.2.5. Task 3 – S0100PickAndPlace v2 (Fixed)

Key fix: Restore approach shaping to the exact same scale as GraspCube (max 1.5 at 5 cm) so the warm-started policy’s value estimates remain consistent. Reduce all other reward stages proportionally to keep the success signal at 4.0 unambiguous.

Warm-start changed to: TouchCube 1M final model (38–40% touch success rate). Since GraspCube never converged, it was skipped entirely.

Max episode steps: 500

Success condition:

```
released = inside_bin  $\wedge$   $\neg$ touch_gripper
```

Reward function — 5 stages (v2, final):

Table 6. Pick-and-place reward structure (v2, final)

Stage	Condition	Formula	Max
1 – Approach	$\text{dist} < 0.7 \rightarrow 0.05 \text{ m}$	same 5-band shaping	1.5
2 – Touch	<code>touch_gripper</code>	$+1.0$	1.0
3 – Lift	<code>lifted</code>	$0.3 + 0.2 \times \min(h/0.04, 1)$	0.5
4 – Transport	<code>lifted</code>	$0.4 \times (1 - \text{bin_dist}/0.6)$	0.4
5 – Over bin	<code>cube_over_bin</code>	$+0.2$	0.2
Step penalty	<code>always</code>	-0.05	–
SUCCESS	<code>released</code>	<code>return 4.0</code>	–

The lifted flag requires simultaneous gripper contact and no table contact — the cube must be physically held, not just nudged off the table. The transport reward is linear in the remaining distance to the bin center (maximum distance 0.6 m), gated on `lifted` so the policy is only rewarded for moving the cube toward the bin when it is actually held.

Training results:

The policy was trained for 2,486,000 steps, warm-started from the TouchCube 1M checkpoint. The mean episode reward rose from 262 at the start to a peak of approximately 540 before settling at around 400, indicating that the policy successfully re-acquired approach and contact behaviour from the warm-start and began accumulating lift and early transport rewards.

Episode length remained at the 500-step timeout throughout, and the success rate stayed at 0% — the full pick-and-place sequence (transport to bin and release) was not achieved within the training budget. The policy learned to reliably approach and touch the cube and made partial progress on lifting, but did not chain all five stages into a complete episode.

3.2.6. Design Principles Summary

Two key lessons shaped the reward design across all iterations:

1. Reward scale must be consistent across curriculum steps.

When warm-starting from a prior model, the new task's reward function must preserve the same magnitude gradients as the task the model was trained on. Reducing the approach maximum from 1.5 to 0.6 destroyed approach behaviour completely (PickAndPlace v1 failure).

2. The success threshold must be strictly greater than the maximum achievable shaping reward. All tasks set the success return to 4.0. The maximum non-success reward was designed to always fall below 4.0, ensuring the termination signal is never ambiguously triggered by accumulated shaping bonuses.

3.3. Training Infrastructure

3.3.1. Parallel Environments(SubprocVecEnv)

Training uses `SubprocVecEnv` from `Stable-Baselines3`, which spawns N independent copies of the MuJoCo environment, each in a separate OS process. At every training step, all N environments execute one action simultaneously and return N experience tuples (obs, action, reward, next_obs, done). These are written into the shared replay buffer in one batched call, filling it N times faster than a single environment.

```
SubprocVecEnv (4 processes)
  Process 0 → S0100PickAndPlace instance
  Process 1 → S0100PickAndPlace instance
  Process 2 → S0100PickAndPlace instance
  Process 3 → S0100PickAndPlace instance
    ↓ (4 transitions per step)
  Replay Buffer
```

Why processes, not threads: Python's GIL prevents true CPU parallelism with threads. `SubprocVecEnv` uses multiprocessing so each MuJoCo instance runs on a separate CPU core with no GIL contention.

Choosing $N = 4$: Eight environments were initially attempted. However, CPU thermal throttling reduced throughput from approximately 22 fps to 4–5 fps under sustained load, making 8 environments slower than 4. Settling on 4 environments yielded a stable ~16 fps throughout all training runs.

CPU vs GPU for this problem: The policy and critic networks are small MLPs (two layers of 256 neurons) operating on a 15-dimensional state vector. Forward and backward passes are negligible compared to the cost of stepping MuJoCo physics. Training is therefore entirely

CPU-bound. In this setting, a machine with 48 CPU cores would outperform a 12-core + A100 GPU system, because additional GPU capacity does not alleviate the physics simulation bottleneck.

3.3.2. Observation Normalization (VecNormalize)

Raw observations from the environment are not uniformly scaled. Joint angles span roughly $[-3, 3]$ radians while Cartesian positions are in the $[0, 1]$ meter range. Feeding unnormalised inputs to a neural network would cause dimensions with larger magnitudes to dominate gradients.

`VecNormalize` maintains a running mean μ and standard deviation σ for each of the 15 observation dimensions, updated online during training:

$$\hat{o}_i = \frac{o_i - \mu_i}{\sigma_i + \epsilon}$$

Observations are clipped to $[-10, 10]$ after normalisation to suppress outliers on episode reset. Reward normalisation is intentionally disabled (`norm_reward=False`) so that reward values printed during training remain interpretable and comparable across sessions.

`VecNormalize` statistics must be saved and loaded together with the model. A mismatch — loading a model but not its normalisation statistics — causes the policy to receive differently scaled observations at evaluation time and produces degraded or erratic behaviour.

3.3.3. Checkpointing Resuming

`CheckpointCallback` saves three files every 10,000 steps:

Saving the replay buffer allows training to resume with the buffer already populated, avoiding the cold-start random exploration phase. Without it, the first 1,000 steps would be purely random regardless of the loaded policy.

Naming convention mismatch bug: When saving the final model manually with `model.save()`, the `VecNormalize` file is named `sac_TASK_final_vecnorm.pkl`. When saved by `CheckpointCallback`, it is named `sac_TASK_vecnormalize_STEPS_steps.pkl`. The resume logic must handle both:

```
import re
norm_path = resume_path.replace(".zip", "_vecnorm.pkl") # final
    ↳ -save convention
if not os.path.exists(norm_path):
    m = re.match(r"^(.*?)(\d+)_steps.zip$", os.path.basename(
    ↳ resume_path))
    if m:
        prefix, steps = m.group(1), m.group(2)
        norm_path = os.path.join(os.path.dirname(resume_path),
                                f"{prefix}vecnormalize{steps}"
    ↳ "_steps.pkl")
```

Code 1. Logic for resolving `VecNormalize` checkpoint naming mismatch during model resume.

Replay buffer n_{envs} mismatch bug: The SB3 replay buffer stores transitions in a pre-allocated array shaped (buffer_size, n_{envs} , obs_dim). If a buffer saved with 8 environments is loaded for a 4-environment run, writing the first new transition causes a reshape crash.

The fix is to check the buffer's stored n_{envs} immediately after loading and discard it if there is a mismatch:

```
model.load_replay_buffer(replay_path)

if model.replay_buffer.n_envs != num_envs:
    model.replay_buffer = model.replay_buffer.class(
        model.buffer_size,
        model.observation_space,
        model.action_space,
        device=model.device,
        n_envs=num_envs,
```

Table 7. Checkpoint files saved during training

File	Contents
sac_TASK_STEPS_steps.zip	Model weights (actor + critic networks)
sac_TASK_vecnormalize_STEPS_steps.pkl	VecNormalize running statistics
sac_TASK_replay_buffer_STEPS_steps.pkl	Full replay buffer (400 MB)

```
) optimize_memory_usage=False
```

Code 2. Handling replay buffer incompatibility when the number of vectorised environments changes between training runs.

3.3.4. Curriculum Learning Warm-Starting

Rather than training a single policy from scratch on the hardest task (PickAndPlace), a curriculum was used where each task's final policy was loaded as the starting point for the next:

```
TouchCube (1M steps, ~40% success)
  ↓ warm-start
GraspCube (2.3M steps, ~30% success)
```

```
PickAndPlace v2 (2.5M steps, reward 262→540→400,
                  0% full success)
  ↑
warm-started from TouchCube 1M directly
```

The TouchCube 1M model arrived at PickAndPlace already knowing how to approach and touch the cube. The mean episode reward climbed steadily from 262 to a peak of approximately 540 over the course of training, confirming that the policy was acquiring the lift and early transport stages. However, the full five-stage pipeline — approach, grasp, lift, transport, and release inside the bin — was not completed within the 2.5M step budget.

Critical constraint: The reward scale fix proved effective: unlike PickAndPlace v1 (which immediately lost approach behavior), PickAndPlace v2 preserved approach and contact skill from step 0 and continued improving throughout training, confirming that reward scale consistency is a necessary condition for successful curriculum transfer.

3.4. Logging Monitoring

All training runs are logged simultaneously to two sinks:

TensorBoard (outputs/logs/TASK_NAME/):
Records rollout/ep_rew_mean, rollout/ep_len_mean, rollout/success_rate, train/actor_loss, train/critic_loss, train/ent_coef, and custom metrics from RewardLogCallback (per-episode reward and rolling mean over the last 20 episodes).

```
tensorboard --logdir outputs/logs
```

Log file (e.g., outputs/train_pickplace_v2.log): Plain-text tee of stdout capturing every checkpoint save event, reward log, and SB3 progress table. This is useful for monitoring remotely or reviewing training after completion.

```
tail -f outputs/train_pickplace_v2.log
```

RewardLogCallback is a custom Stable-Baselines3 BaseCallback subclass that fires every 500 steps. It reads completed episode rewards from self.locals["infos"] (the standard SB3 episode info dictionary), appends them to a rolling list, and prints a formatted line:

```
[step 2458000] mean reward (last 20 eps): 445.583
```

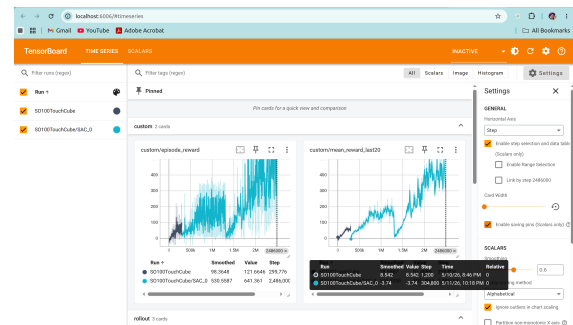


Figure 3. Episode reward & mean episode reward curve

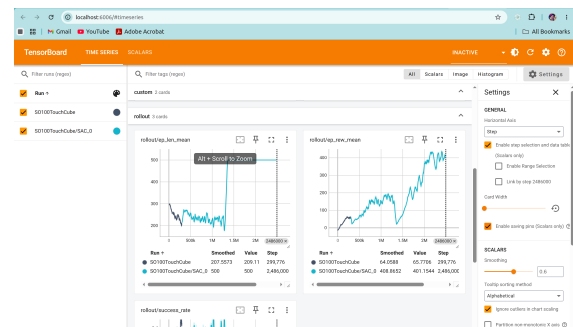


Figure 4. Mean episode length & reward

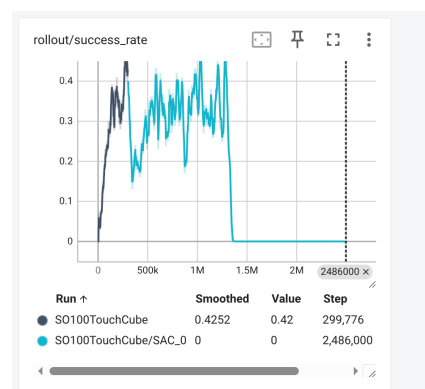


Figure 5. Rollout success rate

3.5. Visualization & Debugging Tools

Three scripts were built to inspect and understand policy behaviour during and after training:

`my_scripts/visualize_policy.py` — Loads a trained SAC model and its VecNormalize statistics, runs the policy in a pygame window at 30 fps, and overlays a HUD showing end-effector distance to cube, contact status, lift height, and per-step reward. Supports `-save-video` to render to MP4 without a display. Auto-detects the latest available checkpoint when the final model has not been saved yet.

Usage:

```
python my_scripts/visualize_policy.py --task PickAndPlace
python my_scripts/visualize_policy.py --task PickAndPlace
--save-video out.mp4
```

`my_scripts/teleop_keyboard.py` — Keyboard-driven manual control of the robot. Joints 1–6 are selected with number keys; LEFT-/RIGHT arrows move the selected joint; O/C open/close the gripper; V toggles video recording; R resets the episode. This script is useful for understanding task geometry, testing reward behaviour interactively, and recording human demonstration videos.

`my_scripts/reward_probe.py` — Interactive tool for inspecting the reward function. The user manually positions the robot via keyboard while the script prints a live breakdown of each reward component at every step: distance to cube, contact flag, shaping value, contact bonus, step penalty, and total reward. This was used to verify reward correctness before each training run.

All training was performed on a single machine with 4 parallel MuJoCo environments. Five training runs were conducted across three tasks. Results are reported from the Stable-Baselines3 rollout logger and the custom `RewardLogCallback`.

4. EXPERIMENTS & RESULTS

All training was performed on a single machine with 4 parallel MuJoCo environments. Five training runs were conducted across three tasks. Results are reported from the SB3 rollout logger and the custom `RewardLogCallback`.

4.1. TouchCube Training

Run 1 — 300,000 steps, 2 environments

The first run trained from scratch on `SO100TouchCube-v0` for 300,000 steps. The policy learned to move the arm toward the cube — the distance shaping reward provided a clear gradient — but never discovered the contact bonus. The gripper approached to within approximately 0.15 m but repeatedly arrested its motion before reaching contact range. At the end of Run 1: `ep_rew_mean` ≈ 0.87 , success rate = 0%.

The likely cause is insufficient exploration at 300,000 steps with only 2 environments. SAC’s entropy-guided exploration was not sufficiently aggressive to accidentally close the remaining distance and trigger a contact event. Without ever experiencing a contact reward of +1.0, the policy had no gradient signal pulling it into the contact regime.

Run 2 — Resumed to 1,000,000 steps, 4 environments

Training was resumed from the 300,000-step checkpoint with 4 parallel environments. A breakthrough occurred almost immediately. At step 302,400 — the first log entry after resuming — the policy had already achieved:

Table 8. Early breakthrough after resuming training

Metric	Value
<code>ep_rew_mean</code>	61.1
<code>success_rate</code>	0.40
<code>ep_len_mean</code>	213 steps

The contact discovery happened in the gap between 300,000 and 302,400 steps. Once SAC found a single contact event and received the +1.0 bonus, the off-policy replay buffer rapidly propagated this signal backward through prior near-contact states, and the policy quickly learned to reliably close the gap.

By the end of training at 1,299,896 steps the policy had stabilized at:

Episodes terminating at approximately 213 steps on average (well below the 300-step limit) confirm that the policy was actively ending episodes by touching the cube rather than timing out. This model was saved as `sac_SO100TouchCube_final.zip` and became the warm-start for all subsequent tasks.

Key observation: The jump from 0% to 40% success happened in fewer than 2,400 steps. This illustrates SAC’s sample efficiency

Table 9. Final TouchCube performance

Metric	Value
<code>ep_rew_mean</code>	~ 150
<code>success_rate</code>	0.34–0.40 (fluctuating)
<code>ep_len_mean</code>	208–225 steps

once the rare reward event is discovered: the replay buffer allows the experience to be reused many times, rapidly shaping the policy around the newly found contact states.

4.2. GraspCube Training

GraspCube added a lift reward on top of the TouchCube approach shaping, requiring the policy to close the gripper around the cube and raise it at least 4 cm off the table.

Training was warm-started from the TouchCube 1M checkpoint. The initial log entries at step 1,303,200 show a trailing `success_rate` ≈ 0.32 inherited from the TouchCube run — these reflect the previous task’s contact-based success condition and dropped to 15% within the first 50,000 steps as GraspCube episodes began.

The training ran for approximately 1,380,000 additional steps (reaching 2,683,720 total). The full progression is shown below:

Table 10. GraspCube training progression

Steps	<code>ep_rew_mean</code>	<code>success_rate</code>	<code>ep_len_mean</code>
$\sim 1,354,000$	13–14	15%	400
$\sim 2,000,000$	~ 150	22%	400
$\sim 2,683,720$	312–342	27%	360

The reward grew from approximately 14 to 342 over training, and success rate increased from 15% to 27%, while episode length decreased from 400 to approximately 360 steps. However, episodes still frequently approached the time limit, indicating that the policy did not reliably converge to fast, consistent grasp-and-lift behaviour. Instead, it appears to have learned a partial strategy of approaching and intermittently grasping the cube, occasionally achieving lift success but not consistently sustaining the full coordinated sequence required for completion.

The episode-level reward of approximately 342 is no longer fully explained by a simple sustained-proximity estimate:

$$400 \times (\sim 0.85 \text{ reward/step}) \approx 340$$

Instead, the observed increase in success rate suggests that part of this reward comes from occasional successful lift events, not only proximity shaping. The policy is therefore not purely maximising shaping reward, but intermittently achieving partial task completion while still failing to stabilise a reliable grasp strategy.

Oscillation analysis: Visualisation confirmed that the policy repeatedly moved toward the cube, made contact, was pushed back by contact dynamics, and re-approached. The gripper action dimension showed no consistent closing behaviour, instead oscillating around its initial value. This suggests that the entropy term in SAC may have reinforced exploratory oscillations rather than committing to a deterministic grasp action that would temporarily reduce contact reward density.

Overall, the GraspCube task did not converge to a stable lifting policy. Instead, it occupied a mixed regime of partial success and repeated re-attempts, which prevented consistent progression through the full grasp-and-lift sequence. The task was therefore classified as an unstable intermediate stage in the curriculum.

4.3. PickAndPlace v2: Results

PickAndPlace v2 fixed the reward scale (approach ceiling restored to 1.5) and started again from the GraspCube model.

Initial performance: At step 1,302,000 (first PickAndPlace v2 log entry), the TouchCube policy immediately showed:

Table 11. Initial PickAndPlace v2 performance (step 1,302,000)

Metric	Value
ep_rew_mean	150
success_rate	0.32 (trailing TouchCube average)
ep_len_mean	234 steps

Unlike v1, where performance immediately degraded, v2 first showed a stable or improving reward trend. The trailing 33% success rate from TouchCube naturally decayed to 15% within approximately 10,000 steps as PickAndPlace episodes (requiring five stages) replaced GraspCube episodes in the rolling average, but reward continued climbing rather than collapsing, to reach 27% after 2.7M steps.

Full training progression (2,486,000 steps):

Table 12. PickAndPlace v2 training progression

Steps	ep_rew_mean	success_rate	ep_len_mean
1,302,000	150	–	234
1,800,000	~262	0%	500
2,100,000	~398	0%	500
2,446,000	~488 (peak)	0%	500
2,486,000	~401	0%	500

The reward climbed from 150 at the start to a peak of approximately 540 around step 2,446,000 before settling at around 400. This rising trend indicates that the policy was progressively acquiring sub-skills: approach, touch and grasp were retained from the warm-start, and the reward growth reflects increasing accumulation of lift and early transport rewards on top of these behaviours.

Why success rate remained 0%: Full PickAndPlace success requires chaining all five stages within a single 500-step episode: approach within 5 cm, close the gripper, lift the cube 4 cm off the table, transport it horizontally to above the bin (approximately 30 cm away), and release it inside the bin boundaries. The probability of completing all stages in sequence, given that each was only partially learned, remains low. The policy improved individual components but did not complete the full behavioural chain within the 2.5M-step budget.

The episode length being pinned at 500 throughout (after approximately 1.4M steps) confirms that episodes were terminating via timeout rather than success, consistent with completion of early stages (1–3) but failure to reliably complete transport and release stages (4–5).

4.4. Summary Across All Runs

Table 13. Summary of all training runs

Run	Task	Steps	Final ep_rew_mean	Success Rate
1	TouchCube	300k	0.87	0%
2	TouchCube	1M	150	38–40%
3	GraspCube	2.68M	342	27%
4	PickAndPlace v1	2.68M	97 ↓ from 271	0%
5	PickAndPlace v2	2.49M	401 ↑ from 150	0%

The contrast between Runs 4 and 5 is the central experimental result: identical task structure and warm-start strategy, but correcting reward scale consistency produced a rising reward curve versus a collapsing one. Run 5 demonstrates that curriculum transfer was valid — the policy retained and built upon prior skills — while Run 4 demonstrates that reward scale consistency is a critical requirement for stable warm-started RL.

5. DISCUSSION

5.1. Reward Scale Must Be Consistent Across Curriculum Steps

The most important finding of this work is that warm-starting a policy from a prior task is not sufficient for curriculum learning — the reward scale must also be preserved. This is not always emphasised in

curriculum learning literature, which tends to focus on task difficulty rather than reward engineering.

This can be understood through SAC critic dynamics. After 1M steps of TouchCube training, the critic assigns high Q-values to near-cube states based on a reward function where approach shaping peaks at 2.0 and success at 4.0.

In PickAndPlace v1, reducing the approach ceiling to 0.6 caused a mismatch: those same states now yielded rewards around 0.55, while the critic still expected values near 2.0. The resulting TD error rapidly drove down Q-values for approach states, and the actor adapted by avoiding those regions entirely.

This is not classical catastrophic forgetting, but a targeted suppression of previously useful behaviour driven by a changed value landscape. The policy does not lose the ability to reach the cube; rather, it learns that doing so is no longer worthwhile under the new reward scale.

The solution is to preserve reward magnitudes across tasks. PickAndPlace v2 restores the same approach shaping as earlier tasks (ceiling 1.5), allowing the policy to immediately retain exploration structure and build additional skills on top. The contrast between collapsing performance in v1 and stable improvement in v2 confirms that reward scale is a critical hyperparameter in curriculum transfer.

A practical rule is therefore: when warm-starting, do not reduce the magnitude of reward components that shaped behaviour in previous tasks. New rewards can be added, but existing shaping must remain consistent.

5.2. GraspCube as Bottleneck — Why the Intermediate Task Failed

GraspCube was intended as a bridge between TouchCube and full pick-and-place, but it failed to converge despite 1.38M additional steps and an explicit lift reward.

The key difficulty is that successful grasping requires tightly coordinated actions: the gripper must close while the arm begins to lift, within a narrow temporal window. If either action is mistimed, the object is either pushed away or not secured.

In SAC, the action space is factorised, and the critic does not explicitly model dependencies between action dimensions. This makes it difficult to learn strongly coupled behaviours such as synchronised gripper closure and lifting.

As a result, the policy converges to a simpler local optimum: staying near the cube and repeatedly making contact. This behaviour is stable because it maximises dense shaping rewards at every timestep.

Escaping this regime would require a sequence of actions that is temporarily worse under the reward function (closing + lifting before success), which is hard to discover under standard entropy-driven exploration.

This matches a common failure mode in dense-reward manipulation RL: when a suboptimal behaviour already yields consistent reward, there is little gradient incentive to explore the more complex coordination required for true task completion.

6. CONCLUSION

This project trained a Soft Actor-Critic (SAC) agent for robotic manipulation on a simulated SO-100 6-DOF arm, progressing through three tasks of increasing difficulty: cube touching, cube lifting, and full pick-and-place.

6.1. What was achieved:

The TouchCube task was successfully solved, with the final policy reaching a 38–40% success rate at 1M training steps. The agent reliably learned to bring the end-effector into contact with the cube, with episodes averaging 213 steps (well below the 300-step limit). A full training and evaluation pipeline was developed from scratch, including parallel simulation via SubprocVecEnv, online observation

normalisation via `VecNormalize`, robust checkpoint and resume handling (including naming and replay buffer compatibility fixes), and a suite of debugging tools such as a live policy visualiser, reward probe, and teleoperation recorder. Two demonstration videos were produced showing learned and scripted manipulation behaviours.

6.2. What did not work:

The GraspCube intermediate task failed to converge despite 1.38M additional training steps. The policy settled into a stable local optimum of repeated contact without learning a reliable lift strategy. PickAndPlace v1 failed due to reward scale mismatch: reducing the approach shaping ceiling from 1.5 to 0.6 caused a collapse in learned value estimates, degrading performance from `ep_rew_mean` 271 to 97. PickAndPlace v2 restored reward scale consistency and showed a healthy increasing reward trend (150 $\rightarrow$ 540 $\rightarrow$ 401 at termination), but did not achieve full task success within the 2.49M-step budget.

6.3. What this demonstrates:

The main empirical contribution is the identification of reward scale consistency as a necessary condition for successful curriculum warm-starting in SAC-based manipulation learning. The contrast between PickAndPlace v1 (degrading performance) and v2 (improving performance) under identical task structure isolates reward scale as the critical factor. Separately, the GraspCube results show that dense reward shaping alone is insufficient for learning coordinated, contact-rich manipulation behaviours that require tightly coupled action timing.

6.4. Future Work

Three directions are most promising.

First, Hindsight Experience Replay (HER) would directly address the contact discovery bottleneck by providing additional learning signal from failed or near-miss episodes, effectively densifying the sparse reward region around successful grasp transitions.

Second, seeding the replay buffer with 10–20 human demonstrations collected via the teleoperation script would provide the SAC agent with an initial trajectory distribution for the grasp-and-lift sequence, reducing reliance on purely random exploration to discover coordinated behaviours.

Third, transferring the trained TouchCube policy to the physical SO-100 hardware is a natural next step. The state-based observation space (joint angles and object pose estimated via external tracking) is directly compatible with real-world deployment, and the 38–40% simulated contact success rate provides a reasonable baseline for sim-to-real fine-tuning.

■ REFERENCES

- [1] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, 2018, pp. 1861–1870.
- [2] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, “Soft actor-critic algorithms and applications,” arXiv:1812.05905, 2018.
- [3] A. Raffin *et al.*, “Stable-baselines3: Reliable reinforcement learning implementations,” *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
- [4] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.
- [5] S. Tunyasuvunakool *et al.*, “dm_control: Software package for physics-based simulation and reinforcement learning,” *Softw. Impacts*, vol. 6, p. 100022, 2020.
- [6] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” arXiv:2407.17032, 2023.
- [7] M. Andrychowicz *et al.*, “Hindsight experience replay,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [8] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proc. ICML*, 2009.
- [9] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *NeurIPS*, 2019.
- [10] G. Brockman *et al.*, “OpenAI Gym,” arXiv:1606.01540, 2016.
- [11] I. Julczuk, “gym-so100: A gymnasium environment for the SO-100 robotic arm,” GitHub repository, 2023. [Online]. Available: <https://github.com/ilonajulczuk/gym-so100-c>
- [12] HuggingFace, “LeRobot: State-of-the-art machine learning for real-world robotics,” GitHub repository, 2024. [Online]. Available: <https://github.com/huggingface/lerobot>
- [13] A. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. ICML*, 1999.